Suivi des Investissements — Présentation technique

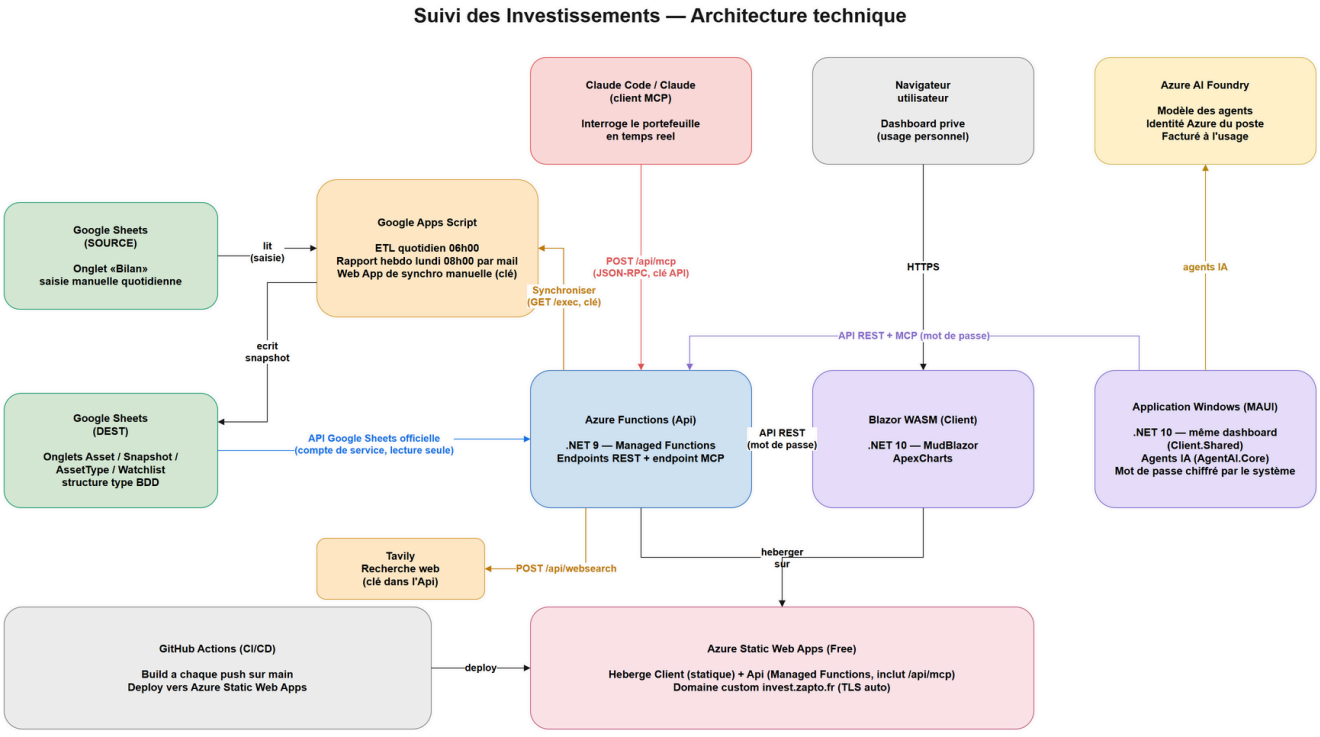
Dashboard personnel de suivi de portefeuille d'investissement multi-supports.

1. Objectif

Visualiser un portefeuille d'investissement multi-supports (actions, obligations, épargne, immobilier, cryptomonnaies, prêts participatifs...) selon deux niveaux de lecture :

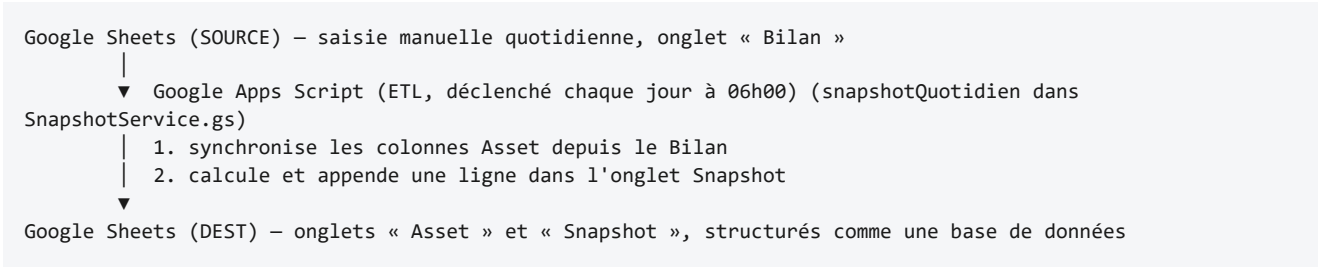
- **Portefeuille** — état du portefeuille au jour le jour : allocations par support, par type d'actif, par actif, par zone géographique.
- **Suivi** — évolution de la performance du portefeuille dans le temps, comparée aux références LifeStrategy 40 et MSCI World et échéancier obligataire

2. Architecture technique



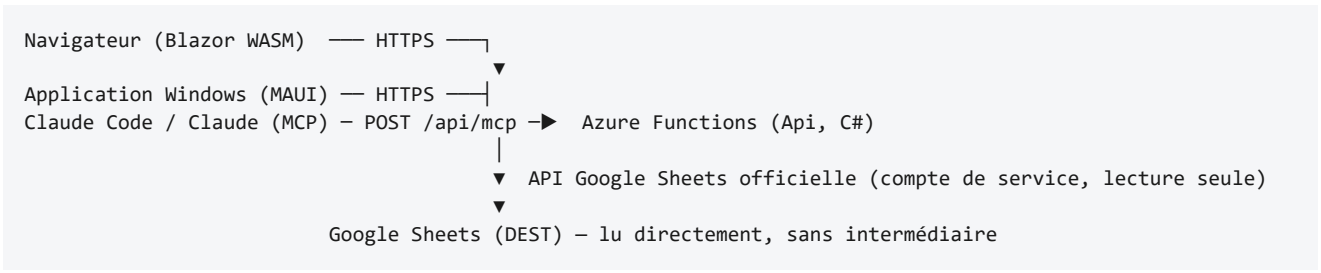
2.1 Flux de données

Alimentation (écriture)



Un rapport hebdomadaire (email HTML) est envoyé chaque lundi à 08h00, généré par le même Apps Script. (rapportHebdomadaire dans WeeklyReportService.gs)

Lecture (dashboard et intégration IA)



L'Apps Script ne fait qu'écrire (ETL + rapport hebdomadaire) ; son seul point d'entrée HTTP est un Web App protégé par clé, qui relance l'ETL depuis le bouton « Synchroniser » du dashboard. L'Api lit le Sheet directement via l'API Google Sheets officielle — pas de cold start, pas d'intermédiaire.

2.2 Composants

Composant	Rôle
Google Sheets	Source de vérité — deux feuilles : `SOURCE` (saisie manuelle) et `DEST` (structurée type BDD, onglets `Asset`, `Snapshot`, `AssetType`, `Watchlist`)
Google Apps Script	ETL quotidien (06h00) + rapport hebdomadaire (lundi 08h00) — écriture uniquement ; appelé de l'extérieur pour la seule synchronisation manuelle
Azure Functions (Api)	Backend serverless C# — lit le Sheet via un compte de service dédié, expose des endpoints REST + un endpoint MCP pour l'intégration avec les assistants IA (Claude)
Blazor WebAssembly (Client)	Dashboard — ne détient aucune clé API ni donnée sensible, consomme uniquement les endpoints de l'Api
Azure Static Web Apps	Hébergement gratuit du Client + de l'Api (Managed Functions), domaine personnalisé `invest.zapto.fr`
GitHub Actions	CI/CD — build et déploiement automatique à chaque push sur `main`

2.3 Point notable — intégration MCP

L'Api expose un endpoint `POST /api/mcp` (protocole JSON-RPC 2.0, Model Context Protocol) permettant à un assistant IA (Claude Code, Claude Desktop, Claude Web) d'interroger le portefeuille en langage naturel — 9 outils exposés (actifs, répartition par classe d'actif, historique de performance, répartition géographique, watchlist...), avec la même logique métier et les mêmes données que le dashboard.

2.4 Application Windows et agents IA

Une application MAUI Windows (`Maui/`) reprend le dashboard et exécute des agents IA sur le poste du propriétaire (`AgentAI.Core/`, Microsoft Agent Framework et Azure AI Foundry), car ils dépassent la limite de 45 s du proxy Static Web Apps. Le workflow d'analyse lance en parallèle l'allocation, le contexte macro-économique et les répartitions, puis un agent de synthèse produit un rapport structuré, affiché dans la page Analyse.

3. Stack technique

Couche	Technologie	Justification
Données	Google Sheets	Écosystème déjà en place, gratuit
ETL	Google Apps Script (triggers temporels)	Gratuit, suffisant pour un traitement quotidien
Backend	Azure Functions, .NET 9 (isolated worker)	Serverless, gratuit, lié nativement à Static Web Apps
Accès données	`Google.Apis.Sheets.v4` (compte de service, lecture seule)	Lecture directe, rapide, sans cold start
Frontend	Blazor WebAssembly, .NET 10	Langage maîtrisé par le développeur (C# de bout en bout)
UI	MudBlazor	Composants riches, bien maintenus
Graphiques	ApexCharts for Blazor	Couvre tous les types de visualisation requis
Hébergement	Azure Static Web Apps (plan Free)	Gratuit, intégration native des Azure Functions
CI/CD	GitHub Actions	Intégration native avec Azure Static Web Apps
Application Windows	.NET MAUI, `BlazorWebView`, .NET 10	Même dashboard que le site, agents IA sur le poste
Agents IA	Microsoft Agent Framework, Azure AI Foundry	Exécutés hors de l'Api, à cause de la limite de 45 s du proxy
Tests	xUnit + Moq (Api, Maui), xUnit + bUnit (Client)	Couverture systématique des services et composants

Architecture applicative : MVVM côté Client (Views / ViewModels / Model / Shared / Services), architecture en couches côté Api (Functions → Services → Mappers), modèles partagés entre les deux projets.

4. Sécurité

- **Accès au dashboard** : protégé par un mot de passe unique (`DashboardAuthMiddleware`), vérifié sur toutes les requêtes de l'Api sauf l'endpoint MCP (protégé par sa propre clé ou ce mot de passe).

Fail-safe : clé ou mot de passe non configurés = accès refusé par défaut.

- **Identifiants Google** (email + clé privée du compte de service) : stockés uniquement dans les App Settings Azure, chiffrés au repos, jamais exposés côté client ni dans le code source.
- **Droits du compte de service** : accès **lecteur uniquement** sur le Google Sheet — il ne peut jamais modifier les données, seul l'Apps Script (compte Google propriétaire) écrit.
- **Endpoints Azure Functions** : liés à Azure Static Web Apps (Managed Functions), joignables par son proxy depuis Internet — la protection vient de l'authentification ci-dessus, pas d'une restriction réseau.
- **Session utilisateur** : expiration glissante d'une heure côté navigateur, mot de passe mémorisé dans le `localStorage` (site) ou chiffré par le système (application Windows).

5. Coûts

Service	Plan	Coût mensuel
Azure Static Web Apps	Free	0 €
Azure Functions	Incluses (Static Web Apps Free)	0 €
Google Sheets API	Gratuit	0 €
Google Apps Script	Gratuit	0 €
GitHub Actions	Gratuit (dépôt public)	0 €
Nom de domaine	Déjà possédé	0 €
Tavily (recherche web des agents)	Gratuit (1 000 recherches/mois)	0 €
Azure AI Foundry (agents)	Facturé à l'usage	Selon les tokens
Total		**0 €/mois hors tokens Foundry**

Document généré à partir de la documentation technique du projet (CLAUDE.md racine, Api, Client, Maui, Scripts).